AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

PROJECT WALKTHROUGH | BEGINNER EDITION

Adaptive Pesticide Spraying with Deep Reinforcement Learning

A clear, step-by-step explanation of the project—from the first data file to the final evaluation

What this guide is for

This guide assumes you are new to reinforcement learning and explains the project in plain language. It describes what each script does, what data it uses, how the simulator works, what the reported results mean, and which claims need careful wording.

The project in one sentence

We built a simulated soybean field where an AI agent observes imperfect signs of rust, weather, crop condition, and remaining spray, then learns when to spray none, a little, a medium amount, or a high amount—while balancing modeled crop-health benefit against pesticide use, unnecessary treatment, and drift penalties.

The most important truth

This is a research prototype evaluated inside a computer simulator. Some inputs are real (drone-image-derived sensor scores and hourly weather), but infestation growth, crop damage, pesticide response, and final "yield" are simulated assumptions. The results are not proof of real-world farm efficacy or safety.

Prepared from the submitted project archive, source files, tables, and figures. The guide separates measured project outputs from assumptions and source-code inconsistencies so you can explain the work accurately.

Contents

- 1. Start with the simple story
- 2. The basic words you need to know
- 3. What is real data and what is simulated?
- 4. The whole project at a glance
- 5. What we did, step by step (Steps 1–13)
- 6. Inside the farm simulator
- 7. How the reward tells the agent what matters
- 8. How PPO and DQN learn
- 9. How the experiments were evaluated
- 10. Results—read them without overclaiming
- 11. What the graphs mean
- 12. Important limits and items to verify
- 13. How to reproduce the workflow
- 14. A simple presentation script and likely questions
- 15. Project file map and glossary

How to use this guide

Read Sections 1–5 first for the story and workflow. Sections 6–8 explain the technical core. Sections 9–12 help you defend the results honestly. Sections 13–15 help you run, present, and navigate the files.

1. Start with the simple story

Imagine a field divided into small squares. Some squares may have more disease than others. A sprayer moves through the field one square at a time. It cannot see the true disease level perfectly; it receives noisy sensor-like readings. Weather can also prevent spraying, and the spray tank is limited.

The computer program repeatedly asks the agent: “Given what you can currently observe, what should you do at this cell?” The agent can choose no spray, low spray, medium spray, or high spray. After the action, the simulator updates the disease, crop health, chemical residue, tank, and reward. The agent practices many simulated seasons and adjusts its strategy to earn more reward.

The research question

In everyday words

Can a learning agent make more useful, resource-aware spray decisions than simple rules when the field is partly hidden, observations are noisy, and weather changes?

“More useful” is defined by a mathematical reward—not by money, actual kilograms harvested, or measured environmental impact. That distinction matters whenever you explain the conclusion.

The five-part mental picture

Part	Plain meaning	Project example
Field / environment	The world the agent is acting in	An 8 × 8 grid of simulated crop cells
Observation	What the agent is allowed to see	Noisy sensor scores, current-cell health, weather, tank, time and location
Action	What the agent chooses	One of four spray levels
Reward	A score that encourages useful choices	Health change minus chemical, waste and drift penalties
Policy	The agent's learned decision rule	A trained PPO model mapping the observation history to a dose

2. The basic words you need to know

Word	Simple explanation	In this project
Reinforcement learning (RL)	Learning by trying actions and receiving scores	The agent practices simulated spraying decisions
Agent	The learner / decision-maker	PPO or DQN choosing spray doses
Environment	The world that responds to the agent	The farm simulator
State	Everything that is true in the world	All cells' infestation, health and residue, weather, tank and time
Observation	The part of the state shown to the agent	A noisy, local, numerical view—not the full field map
Action	A choice the agent can make	None, low, medium or high dose
Reward	A score after a choice	A scalar that combines crop-health change and penalties
Episode / season	One complete run from start to finish	Three field sweeps, 192 agent decisions
Policy	A rule that maps observations to actions	The trained PPO network or a hand-written threshold rule
Baseline	A simpler comparison method	No-spray, fixed-dose, or threshold policies
Seed	A number that controls random draws	Different simulated seasons and separate training runs use different seeds
Generalization	Working in conditions unlike training	Testing an edge infestation pattern not used by default during training

Word	Simple explanation	In this project
Ablation	Removing/changing one design ingredient	Comparing reward-shaping weights
Hyperparameter	A setting chosen by the designer	Learning rate, network size, batch size or shaping weight

Important distinction: model seed vs season seed

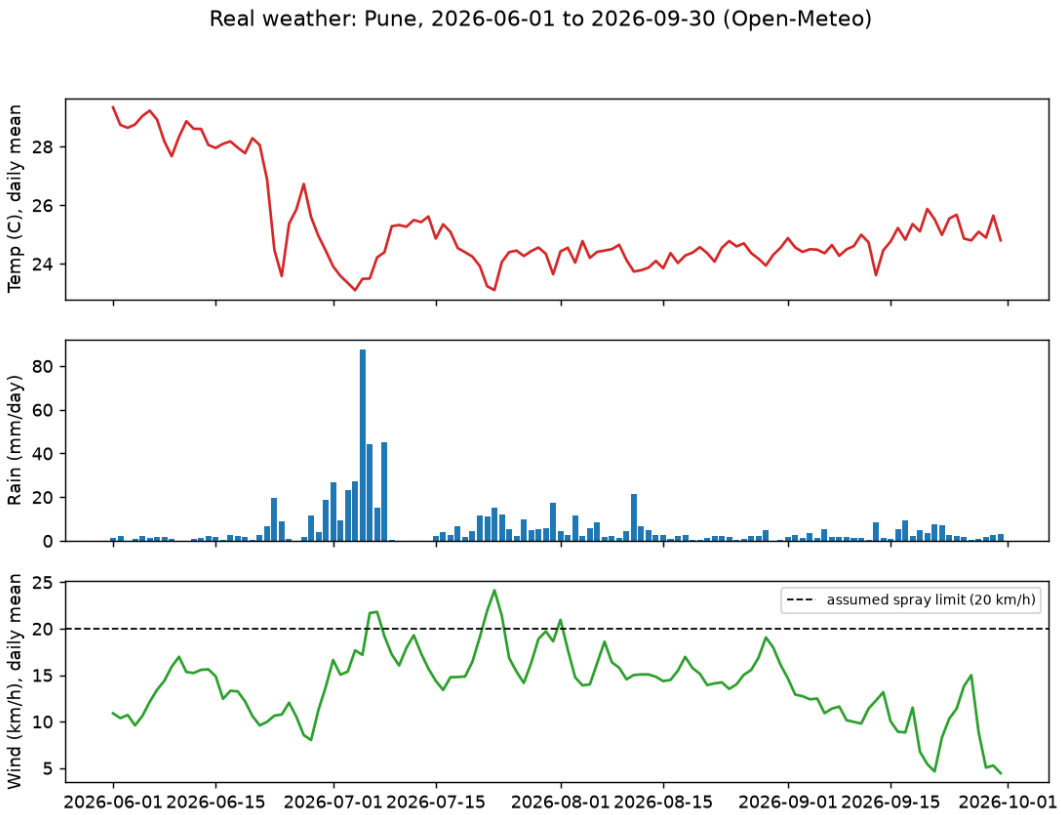
A training seed changes the randomness used while a PPO/DQN model learns. An evaluation seed creates a particular simulated season. Three PPO training seeds means three independently trained policies; testing one policy on 100 seasons means 100 evaluation runs. These are different kinds of repetition.

3. What is real data and what is simulated?

The project deliberately mixes empirical inputs with a custom simulator. Do not describe the entire farm environment as "real data." A clear explanation is: "We use real imagery-derived sensor-score samples and real hourly weather, but the pest and crop dynamics are simulated using assumed equations."

Component	Source / status	What it can support
Soybean images	Real UAV/aerial images from the MH-SoyaHealthVision dataset are referenced by the preprocessing code. The archive contains derived tile features/scores, but the inspected copy did not include the original raw image folders.	Image-colour features and held-out classifier score distributions, if the original source data and split are available.
Weather	Hourly temperature, rain and wind CSVs from Open-Meteo for Pune and Thiruvananthapuram, June 1–September 30, 2026.	Weather variation in the simulator; it does not validate all possible farm climates.
Sensor model	A binary healthy-versus-rust image-cell classifier; score samples from held-out cells are reused in the simulator.	A noisy "what the agent sees" signal based on real-score distributions. It is not a live camera inside a real farm.
Pest / disease path	Procedurally created infestation that grows and spreads according to hand-coded equations.	Controlled simulated scenarios only; not measured disease progression.
Crop health / yield	Cell health is updated by the simulator. Final yield is mean cell health × 100.	A simulator metric, not harvested mass, measured crop yield, or farm revenue.
Spray effect / residue	Hand-coded dose response, residue buildup/decay, and rain washing.	Behavior under chosen equations; not proof of real pesticide efficacy or persistence.

Component	Source / status	What it can support
Safety and costs	Configured wind/rain gates and reward weights. Several are explicitly assumptions.	Comparisons under those chosen values; not legal, label, agronomic, or environmental guidance.



Project figure: hourly/daily weather context. Real weather is fed into the simulated farm; pest dynamics remain modeled.

Why this distinction is not a weakness to hide

It is normal for a simulation project to use assumptions. The scientific job is to label them, test sensitivity, compare policies consistently, and avoid claiming more than the model can show.

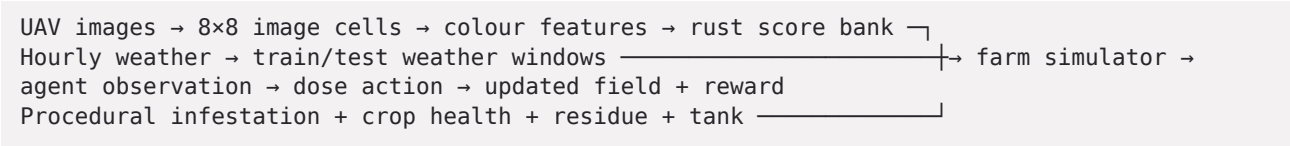
4. The whole project at a glance

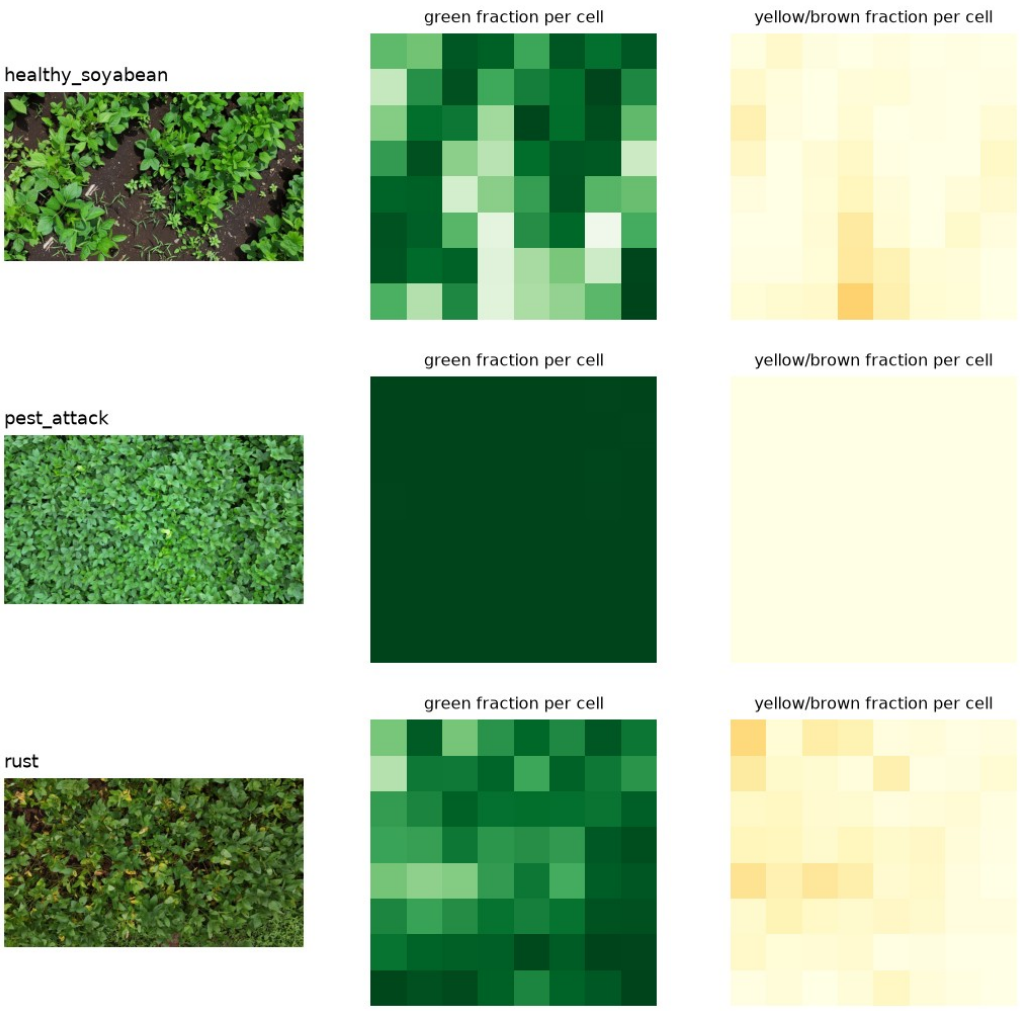
The project pipeline moves from images and weather, to a sensor score bank, to a farm simulator, to training and evaluation. The agent never trains directly on a real spray robot or live farm field.

Stage	Input	What is made	Why it is needed
Image pipeline	UAV soybean photos + class folders	Cell-level colour features and held-out rust-probability scores	Gives the simulator realistic-looking noisy observations
Weather pipeline	Hourly city weather from Open-Meteo	Temperature, rain and wind rows with train/test time ranges	Makes conditions change across episodes

Stage	Input	What is made	Why it is needed
Simulator	Score bank + weather + assumed dynamics	An 8×8 simulated field and Gymnasium step/reset interface	Lets an agent practice many seasons safely and cheaply
Baselines	Simulator + hand-written policies	No spray, fixed spray, thresholds, oracle reference	Shows how simple strategies behave before RL
Training	Training-weather episodes	PPO and DQN model checkpoints and learning logs	Learns an adaptive observation-to-dose policy
Evaluation	Held-out seeds/scenarios + fresh episodes	Tables, curves, trade-off and robustness comparisons	Tests whether a policy behaves well beyond its training episodes
Demo / visualization	Saved simulator data and HTML files	Interactive/demo artifacts and report charts	Makes the system easier to inspect; generator portability should be checked

Data flow in one line





Project figure: one drone image is divided into 8×8 cells and converted into cell-level maps/features.

5. What we did, step by step (Steps 1–13)

The numbered scripts are the project's analysis notebooks saved as Python files. Their "# %%" markers divide a script into cells. Some stages only prepare inputs; later stages perform the longer training runs.

Step 1 — Check the computer setup (`setup.py`)

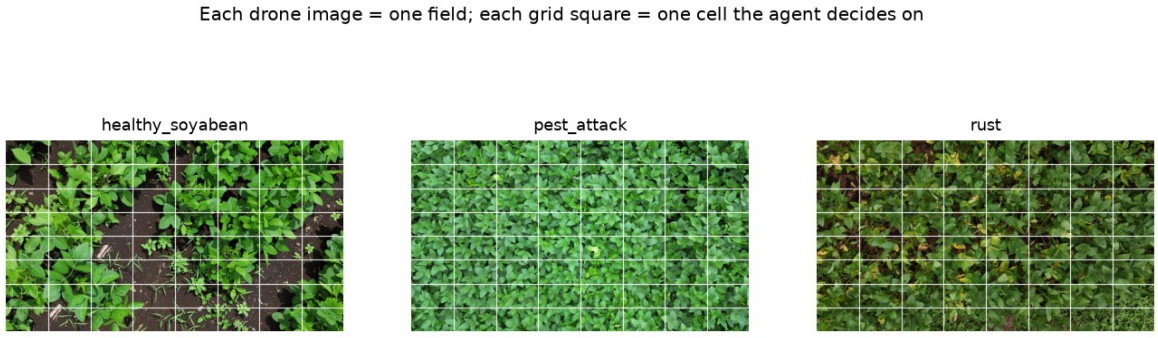
This is a readiness check, not the AI training itself. It checks the Python version, disk space, and core libraries such as NumPy, pandas, Matplotlib, Pillow and Gymnasium; creates project folders; and tests that the free Open-Meteo geocoding service can be reached.

- Expected purpose: confirm that the machine can run the data/simulation pipeline and create the needed folders.
- It does not download the soybean image dataset or train any model.

Step 2 — Prepare the drone-image dataset (`field_dataset.py`)

The code expects the downloaded UAV portion of the MH-SoyaHealthVision data under `data/raw/`. It searches class folders such as healthy soybean, rust and pest attack. The folder name becomes the image label. It samples up to 150 images per class, resizes the longest image edge to at most 1,024 pixels, saves cleaned JPEG copies, and creates an example 8×8 overlay.

- Why: consistent image files make later cell splitting and feature extraction easier.
- The derived data are in the archive, but the inspected copy does not include the original `data/raw/` / `data/field_images/` source directories. To rebuild this step fully, the original image dataset must be available.



Project figure: an example soybean image with the simulator's cell grid overlaid.

Step 3 — Obtain real weather (`weather.py`, `src/weather_api.py`)

The weather helper geocodes a city name and requests hourly temperature, precipitation and wind speed from Open-Meteo. The project includes Pune and Thiruvananthapuram CSVs for June 1 through September 30, 2026. The file is cached so a later run can reuse it instead of requesting it again. Missing numeric observations are interpolated and filled.

- The scripts split the weather rows chronologically: the first 70% is the training range; the last 30% is the later test range.
- The configured wind and rain limits are assumptions. Weather observations tell the simulator what conditions occurred; they do not determine what a pesticide label permits.

Step 4 — Turn images into grid-cell features (`field_grid.py`)

Each resized photo is split into 8 rows × 8 columns. The script calculates eight visual features per cell: average red, green and blue, average and variation of excess green, and the fractions of green, yellow/brown and dark pixels. All cells from one image stay together in the same split. The split is made in consecutive image blocks (nominally 70% train, 15% validation, 15% test) to reduce leakage from adjacent or overlapping drone frames.

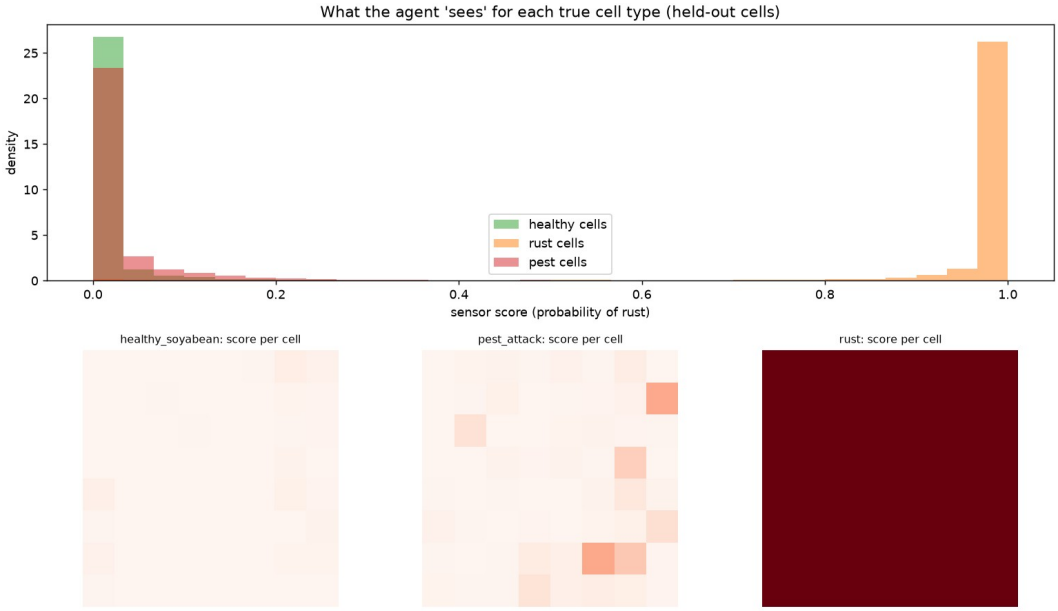
- Main tables: `data/tiles/tile_features.csv` and `data/tiles/image_index.csv`.
- Why: a cell-level table is easier for a small classifier to learn from than raw images.

Step 5 — Build the "sensor" classifier (`sensor.py`)

The selected classifier distinguishes healthy cells from rust cells using those eight colour features. It tests standardized logistic regression and histogram gradient boosting. It chooses between them by validation-set ROC AUC, then computes test-cell and per-image accuracy/AUC. The pest-versus-healthy checks and feature-removal checks are honesty tests: they help see whether the colour signal is actually disease-related or could be mostly soil/canopy structure.

- Outputs include a saved model, per-cell `cell_scores.csv`, and a held-out `score_bank.npz` with healthy/rust/pest score samples.
- The sensor score means "classifier probability of rust," not a calibrated measurement of infestation severity.

- I found the code that prints classifier metrics, but not a saved text log with the realized numbers. Do not quote an accuracy or AUC unless you rerun the script and capture its actual output.

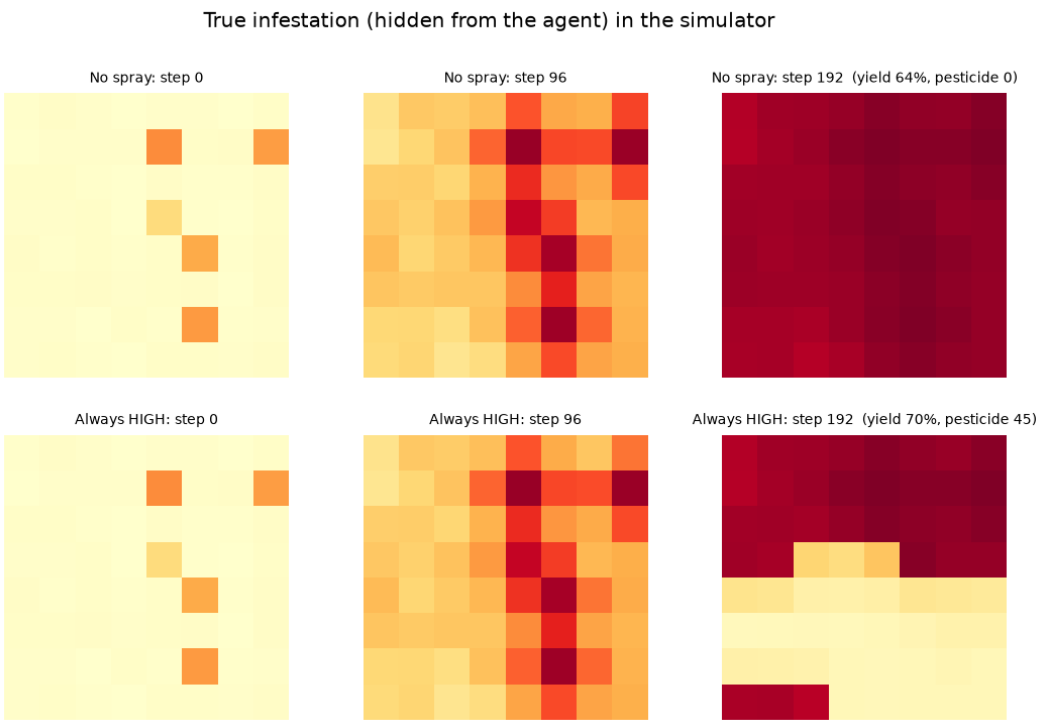


Project figure: held-out score distributions and example score maps. Scores are used as observation samples in the simulator.

Step 6 — Build and sanity-check the farm simulator (`simulator.py`, `src/spray_env.py`)

The simulator combines the real weather table and held-out sensor score bank with a procedural infestation map and hand-coded crop/disease/spray equations. It checks the Gymnasium API, prints example steps, runs simple policies across seasons, draws infestation maps, and examines how weather gates actions.

- `src/spray_env.py` is the environment itself; `simulator.py` is the demonstration and sanity-check script.
- The complete true infestation map and pesticide residue are hidden from the agent; the evaluator can still inspect them for summaries.



Project figure: simulated field state/maps under different policy behavior. These are model outputs, not photographed field outcomes.

Step 7 — Establish simple baselines (`baselines.py`)

Before training RL, the project checks whether the task has "headroom"—whether an intelligent rule can improve on doing nothing. Baselines include no spray, fixed medium spray, a simple threshold, a tuned neighbour-aware threshold, and an oracle. The threshold cut-offs are selected only on training-weather seeds 0–19; the first evaluation uses 30 later test seasons (seeds 1,000–1,029).

- The oracle is deliberately not a fair deployable method: it can look directly at the true hidden infestation. It is an upper-reference / "what if the sensor were perfect?" comparison.
- An initial 30-season baseline table reports tuned-threshold gain +1.50 reward points on familiar patterns, but −0.44 on the edge-pattern scenario. It is not universally better than no spray.

Step 7b — Compare tank sizes (`tank_check.py`)

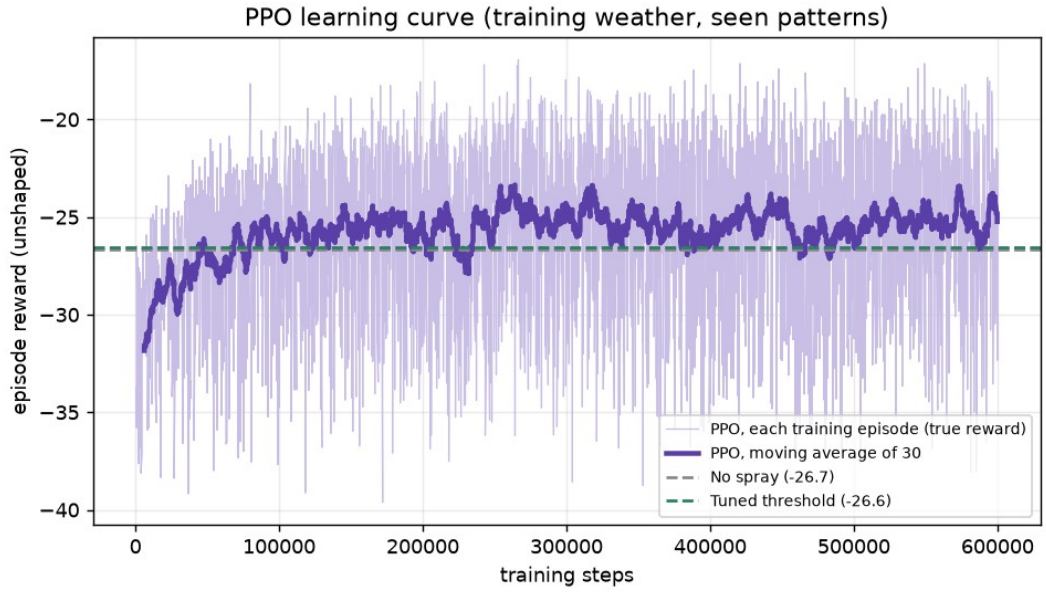
This stage compares 45, 70, 100 and 150 dose units. The goal is to set one resource budget before comparing learned agents, rather than quietly changing tank capacity for each method. Later PPO comparisons use a 100-unit tank.

- Interpretation: 100 is the experiment's chosen setting—not a proven agronomic optimum or a real-world tank recommendation. The tank table shows that a larger tank does not automatically improve reward.

Step 8 — Train PPO (`train_ppo.py`)

PPO means Proximal Policy Optimization. The script trains a neural policy with four parallel simulator environments, tank capacity 100, 600,000 environment steps, and a shaping weight of 0.3 during training. It first runs a 20,000-step smoke test so the user can estimate runtime and find basic setup errors. The model and episode log are saved under `models/` and `outputs/`.

- PPO training weather comes only from the first 70% of hourly rows. Threshold-rule parameters are also tuned on training weather, not on the later test period.
- Important: reported evaluation rewards are the unshaped/base reward even though shaping is added during training.

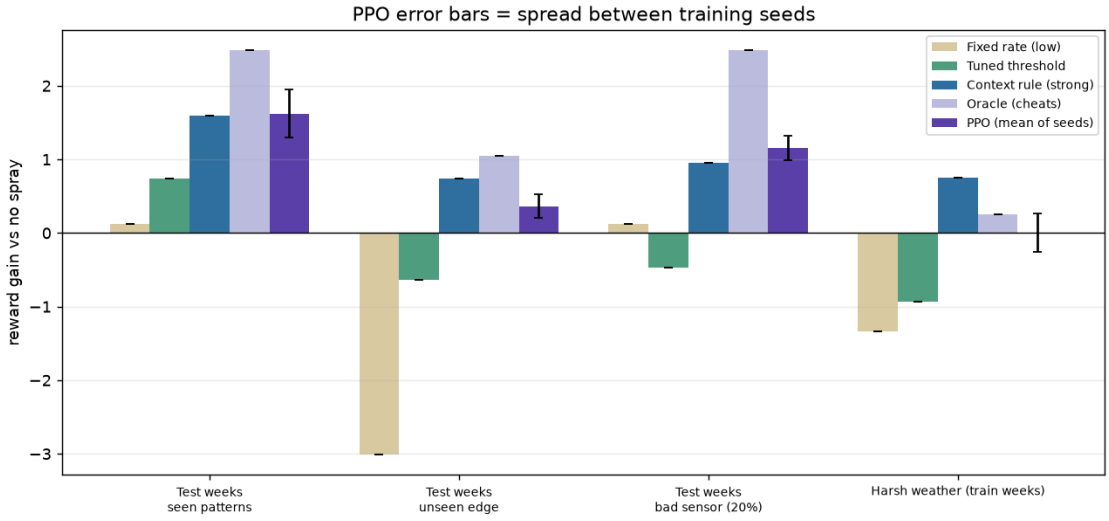


Project figure: training reward as PPO learns. A learning curve shows change over training steps; it does not by itself prove superiority on held-out tests.

Step 9 — Repeat PPO runs and add a stronger rule (`robust.py`)

One training run can be lucky or unlucky, so the project trains PPO with seeds 0, 1 and 2. It also searches for a stronger "ContextRule" that adjusts its spray thresholds according to wind and time left in the season. Evaluation reuses 30 held-out test seeds across familiar patterns, an unseen edge pattern, 20% sensor class-flip noise, and training-week weather.

- On the familiar-pattern 30-season protocol, PPO mean gain is +1.62 (training-seed spread 0.33) and the context rule is +1.59: the supplied summary says there is no clear difference.
- PPO does not lead every case: on unseen-edge PPO is +0.36 versus context +0.75; under the "harsh" training-week weather case PPO averages 0.00 versus context +0.76.



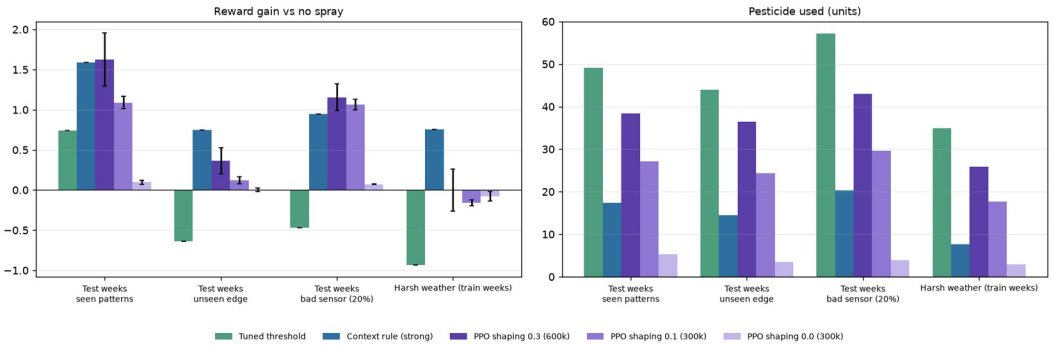
Project figure: multi-seed PPO and heuristic comparison. Read this beside `outputs/step09_summary.csv` and keep it separate from the 100-fresh-season results.

Step 10 — Test reward shaping (`shaping.py`)

Reward shaping adds a training-only hint for reducing total infestation, which may help learning before crop health changes much. The project compares shaping weights 0.3, 0.1 and 0.0. The 0.3 group was trained for 600,000 steps with three seeds; the 0.1 and 0.0 groups used 300,000 steps and two seeds each.

Training shaping	Training budget / seeds	Seen-pattern gain vs no spray	Final health ("yield %")	Pesticide units
0.3	600k / 3 seeds	+1.62	76.57	38.44
0.1	300k / 2 seeds	+1.09	72.32	27.12
0.0	300k / 2 seeds	+0.10	64.76	5.26

The trend is consistent with a trade-off: in these outputs, lower shaping corresponds to less pesticide and also lower health/reward. However, it is not a perfectly controlled causal test because the compared groups differ in training time and number of model seeds.

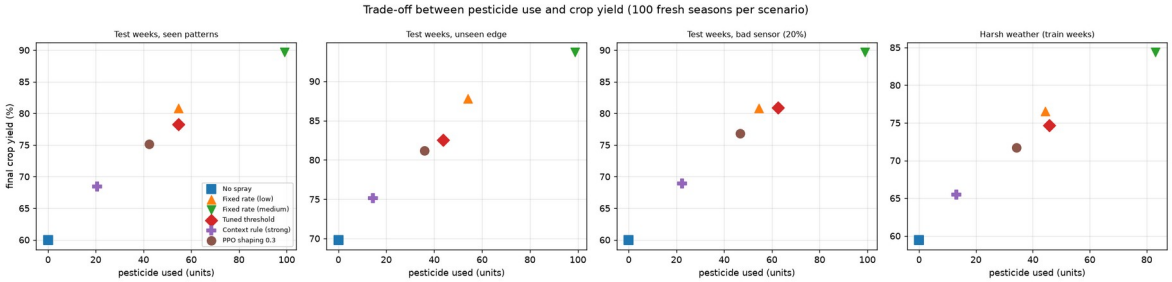


Project figure: reward-shaping comparison. Pair the graphic with its CSV; the training budgets differ by group.

Step 11 — Study yield–chemical trade-offs (`tradeoff.py`)

This is a separate, larger evaluation on 100 fresh seeds (3,000–3,099), described by the script as not used for tuning/model selection/earlier testing. The policies are held fixed. The script records yield, pesticide, unnecessary-spray and drift components, and also recomputes reward under alternative weights without retraining the policies.

This matters because policy ranking depends on what the reward values. A policy can produce higher simulated crop health but still receive worse reward if the additional pesticide penalty is too large.

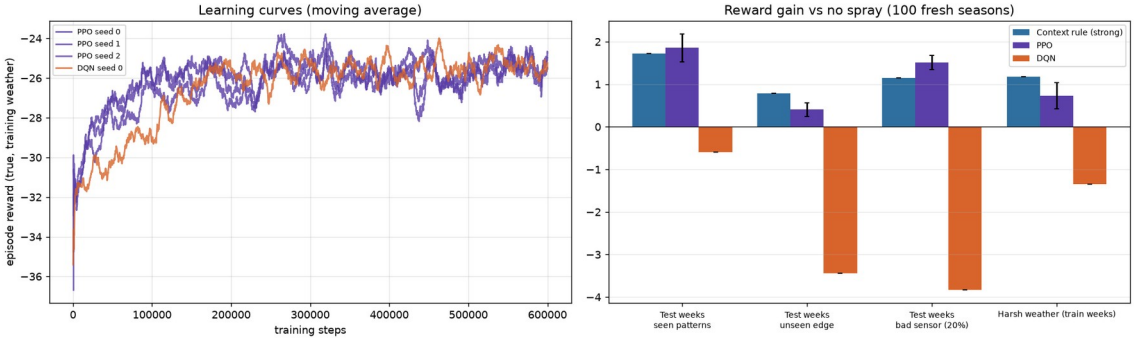


Project figure: crop-health vs pesticide trade-off across policies, from the 100-fresh-season analysis.

Step 12 — Compare DQN (`dqn.py`)

DQN means Deep Q-Network. It learns a value for each of the four dose actions and is trained for 600,000 steps in the same simulator. The visible code uses a replay buffer and a target network, while the PPO code uses multiple parallel environments and a policy-optimization update.

The supplied `step12_table.csv` reports DQN seen-pattern gain −0.60, simulated yield 84.81%, and 81.95 pesticide units. It has higher simulated yield than PPO but much lower reward because chemical use is penalized. Treat this as a reported output, not as an unquestionable repeatable benchmark: the visible `dqn.py` evaluation code calls `SmartThreshold(tau=0.15)`, while the actual `SmartThreshold` class accepts `cuts` and `w`, not `tau`. Also, its evaluation block is a single 100-episode run with seed 123, while the checked-in table is formatted as a multi-scenario table. This should be reconciled before claiming the table was reproduced directly by the current script.



Project figure: PPO/DQN/baseline comparison as provided. Use the source-code note above when explaining the DQN row.

Step 13 — Graphs and interactive demo (`plot_evaluation_graphs.py`, `make_demo.py`)

The project includes several kinds of output: step-specific plots linked to CSV tables, a second set of report-style graphs, and HTML demo artifacts. A graph generator is not automatically proof that every plotted value came from raw experiment logs; its data source must be checked.

- `outputs/demo.html` and `outputs/interactive_farm.html` are included artifacts. Their presence does not prove the generator is portable or was rerun from scratch.
- `make_demo.py` points to an absolute Windows path (`C:/Users/aadib/.../generate_interactive_farm.py`) that is not part of the archive, so it is not portable as supplied.
- `plot_evaluation_graphs.py` hard-codes several chart values; its cumulative-regret panel creates random curves rather than reading measured episode-by-episode regret. It also includes a "no state history" ablation not found in the reviewed experiment CSV/script. Use CSV-backed step plots as primary evidence, or regenerate the report graphs from one documented data table.

6. Inside the farm simulator

The field and the route

The default field is 8×8 = 64 cells. The sprayer follows a fixed back-and-forth (serpentine) route: one row left-to-right, the next row right-to-left. The season has three complete sweeps. A full

season therefore has 64 × 3 = 192 decisions. The code treats a complete sweep as one day and maps real weather readings over the simulated steps.

Hidden state versus visible observation

The hidden state contains true infestation severity, crop health, pesticide residue in every cell, weather, tank level, and time/route position. The agent does not get the full pest map. At each step it receives 13 numbers: five local sensor scores (current cell plus four neighbours), current-cell health, normalized wind, a rain-block flag, normalized temperature, remaining tank fraction, season progress, and row/column position. With history length four, these 13-number observations are stacked into 52 inputs.

A boundary neighbour outside the field is marked −1. At the first step the same initial observation is copied four times to fill the history buffer; it is not four distinct past observations. The agent can use recent observation sequence, but still does not see the true complete field.

From severity to sensor score

For a simulated cell with severity p, the environment first draws whether the sensor class is “rust” with probability p. If yes, it samples one held-out rust score from the real score bank; otherwise it samples one held-out healthy score. Optional sensor-flip noise can reverse the class. So p is a hidden simulator variable; the input feature is a noisy score sampled from the classifier’s score distributions, not a direct p readout.

The four dose choices

Action label	Internal dose	What it means
None	0	Do not spray this cell now.
Low	1/3 dose unit	Small treatment request.
Medium	2/3 dose unit	Intermediate treatment request.
High	1 dose unit	Largest available one-step treatment request.

The code defaults to 45 tank units in the environment class, but the main training/comparison scripts explicitly use a 100-unit tank. A “dose unit” is the simulator’s unit. The project does not provide a verified litres conversion, so do not call it litres.

Weather and hard action blocks

If a positive spray action is requested, the simulator forces it to no spray when wind is above the configured limit, rain is above the configured rain threshold, or the tank cannot cover the dose. In the core source defaults these settings are wind > 20 km/h and rain > 0.2 mm for the weather observation. These limits are documented in code as assumptions, not as professional application guidance. Before your presentation, check that every slide/report uses the same thresholds as the code.

How simulated infestation and crop health change

A spray reduces the current cell's infestation multiplicatively using `exp(−kill × dose)`; higher doses suppress more, with diminishing returns. Spray raises pesticide residue in that cell and subtracts from the tank. Then the simulator updates the whole field: infestation grows and spreads from neighbours; temperature and rain alter the modeled growth; residue slows growth and then decays; rain can wash residue away; and crop health falls in proportion to updated infestation. The relevant settings include growth 0.022, spread 0.012, damage 0.004, kill 3.0, residue decay 0.985, and rain wash 0.4.

These values are the model, not measured biology

The code explicitly labels pest growth/spread/damage, spray response, residue behavior, temperature/rain growth effects, and initial infestation patterns as assumed—not fitted to observed field outcomes. The simulation is useful for controlled comparisons but needs agronomic calibration and field validation before real deployment.

7. How the reward tells the agent what matters

The base per-step reward is:

```
Base reward = w_y × dYield − λ_c × C − λ_u × U − λ_e × E
Project defaults: w_y = 1.0, λ_c = 0.10, λ_u = 0.15, λ_e = 0.15
```

Symbol	Plain meaning in the source code	Why it is in the formula
dYield	Change in the sum of cell health at this step	Rewards preservation of simulated crop health; it is not actual harvest weight.
C	Dose used during this step	Penalizes total chemical consumption.
U	Dose used if pre-spray infestation is below τ = 0.15	Adds a larger penalty for spraying a cell with little modeled infestation.
E	Dose × normalized/clipped wind	Penalizes a simple modeled drift-exposure proxy.

A negative reward does not automatically mean the code failed. The crop-health change can be negative and every spray can add penalties. The useful comparison is often the difference from the no-spray policy under the same scenario. "Gain +1.86" means the policy's mean base reward is 1.86 reward points above that scenario's no-spray mean—not $1.86, 1.86% yield, or a physical unit.

Reward shaping: a training hint

With shaping weight 0.3, the training reward also adds:

```
shaping × (load_before − γ × load_after),   with γ = 0.995
```

Here “load” is the total hidden infestation in the simulator. This extra term can give the learner a more immediate signal before crop health visibly changes. In these scripts shaping is used for training only; `true_reward` and evaluation use the base reward without shaping. Therefore the agent is learning with a training objective that is related to—but not identical to—the score used for reporting.

8. How PPO and DQN learn

PPO: adjust the action strategy gradually

PPO (Proximal Policy Optimization) is a policy-gradient method. Imagine a student trying choices and getting feedback. PPO estimates which choices led to better long-term reward, then adjusts the policy in bounded steps so it does not change too abruptly. The project uses a small neural network to map the 52 observation-history values to the four available actions.

PPO setting	Project value	Plain-language interpretation
Training environment	4 parallel instances	Collects experience from four simulator copies at once.
Total training	600,000 steps	Number of agent decisions used for the full run.
Learning rate	3×10^{-4}	How strongly network weights are adjusted each update.
Rollout length	512 steps per environment	How much recent experience is gathered before updating.
Batch size / epochs	256 / 6	How experience is reused in each update.
Discount factor γ	0.995	Values future rewards; favors long-horizon effects.
GAE λ / clip range	0.95 / 0.2	Settings that stabilize reward estimates and policy changes.
Network	Two 64-unit layers for policy and value	Compact neural network architecture.

DQN: learn the value of each action

DQN (Deep Q-Network) learns an estimated long-term value, called Q, for each of the four possible actions. It uses a replay buffer (a memory of past transitions) and a target network to make learning more stable. The project config uses a 50,000-transition buffer, 1,000 initial learning steps, batch size 64, a 1×10^{-4} learning rate and a [64, 64] network. The visible DQN script uses a single simulator environment, unlike PPO’s four parallel environments.

Why compare two algorithms?

The comparison asks whether the observed performance is specific to PPO or whether another standard deep-RL method behaves similarly. A fair comparison requires the same environment, action meanings, training/test weather split, reward definition, and sufficient repeated training seeds. Differences in code or evaluation protocol must be reported.

9. How the experiments were evaluated

The train/test split

Weather rows are chronological. The first 70% are the training range; the later 30% are the usual held-out test range. Baseline cut-offs are tuned on training-weather seeds. The evaluation uses a new set of random episode seeds. The project also reuses training weeks in a scenario called "harsh weather," so that scenario is a stress test under the training portion—not a fully unseen weather period.

Four main scenario types

Scenario	What changes	What it tests / does not test
Seen patterns	Clustered/random are the usual training patterns; later weather is used	Performance under familiar procedural patterns and later weather rows.
Unseen edge	Initial infestation placed on a field edge	Generalization to one pattern shape the agent did not get by default during training.
Bad sensor (20%)	A sensor class can be flipped with probability 0.20	A controlled synthetic noise stress test—not a measured 20% camera failure rate.
"Harsh weather"	Uses training-week weather rows	Policy behavior under a chosen weather slice; not evidence about a separate future monsoon.

Two result protocols—do not combine them

Protocol	Evaluation episodes / seeds	Main use	Main file
Repeated PPO + robust rule	30 seasons, seeds 1,000–1,029; 3 separate PPO training seeds	Compare PPO variability with stronger context rule across scenario changes.	`outputs/step09_summary.csv`
Fresh trade-off evaluation	100 seasons, seeds 3,000–3,099	Evaluate fixed policies on a larger new episode set and study trade-offs.	`outputs/step11_table.csv`
Reported DQN table	Checked-in table includes multiple scenarios; visible DQN script describes a 100-episode seed-123 evaluation	Use cautiously until the visible script and table provenance are reconciled.	`outputs/step12_table.csv`, `dqn.py`

Why two tables may not match exactly

The weather ranges, random seeds and episode samples differ. Step 7/9/10 use the 30-season evaluation set, while Step 11 uses 100 fresh seasons. A small difference is not automatically a bug, but you must quote which protocol each number came from.

10. Results—read them without overclaiming

Early 30-season baseline check

In `outputs/step07_baselines.csv`, the seen-pattern rows show a no-spray yield of 63.20% and a tuned-threshold yield of 79.13%, using 44.68 pesticide units. Tuned threshold reward gain is +1.50 ± 0.36. However, on the unseen-edge pattern its gain is −0.44 ± 0.19. The oracle gain is +3.71 ± 0.35, but the oracle reads the hidden true infestation and is not a deployable policy.

Repeated PPO training versus the stronger context rule

Scenario (30 seasons)	PPO gain (mean; spread across PPO seeds)	Context-rule gain	What the supplied summary says
Seen patterns	+1.62; 0.33	+1.59	Mixed / no clear difference
Unseen edge	+0.36; 0.16	+0.75	Mixed / no clear difference
20% sensor flips	+1.16; 0.16	+0.95	Mixed / no clear difference
Training-week weather	0.00; 0.26	+0.76	PPO worse across the reported seeds

This is why the defensible conclusion is not “PPO beats every rule.” The robust experiment reports similar performance on familiar patterns and a context-rule advantage in some scenarios.

Larger 100-fresh-season result table

Policy (seen patterns)	Mean simulated health / “yield”	Pesticide units	Reward	Gain vs no spray
No spray	59.94%	0.00	−25.64	0.00
Context rule (strong)	68.44%	20.46	−23.92	+1.72*
PPO, shaping 0.3	75.18%	42.41	−23.78	+1.86
Fixed medium	89.69%	99.05	−33.41	−7.77*
DQN (reported table)	84.81%	81.95	not shown in this table	−0.60**

*Context and fixed-medium gains are reward differences recomputed from the Step 11 table: policy reward minus no-spray reward, rounded. **DQN gain is taken from the separate Step 12 CSV; see the protocol/code caveat. Do not treat the rows as one perfectly controlled single output table.

In the Step 11 table, PPO's reward is only 0.14 points higher than the context-rule reward (−23.78 versus −23.92), while PPO uses 21.95 more pesticide units (42.41 versus 20.46). PPO also preserves more simulated health in this table (75.18% vs 68.44%). Compared with fixed-medium spraying, PPO uses about 57% fewer units (1 − 42.41/99.05) and receives a much better reward, despite its lower simulated health. These are different objective dimensions, not contradictory arithmetic.

The plain-English result

On the project's chosen simulator and reward, PPO improves on no spraying and lands at a health/chemical trade-off. A stronger hand-built rule remains competitive. DQN and fixed spray can produce more simulated crop health while receiving worse reward because they spend much more chemical. No policy is "best" on every metric.

11. What the graphs mean

Figure / file	How to read it	A safe takeaway
`step02_field_grid.png`	One image with an 8×8 grid	Shows how imagery is divided into cells.
`step03_weather.png`	Weather through time	Shows the source weather input; not the simulated disease response.
`step04_tile_maps.png`	Cell colour maps / features	Shows image-to-grid preprocessing.
`step05_sensor.png`	Score histograms and spatial maps	Shows the score distributions used to model observations; only quote measured metrics if logged.
`step06_simulator.png`	Example hidden infestation/field maps	Illustrates how the simulated system evolves under its assumptions.
`step08_learning_curve.png`	Episode reward over training steps	Shows whether PPO reward trends improve during training; test comparisons still matter.
`step09_results.png`	Multi-seed robust comparison	Use alongside `step09_summary.csv`; this is the 30-season protocol.
`step10_shaping.png`	Reward-shaping variants	Shows the trade-off, with unequal training budgets and seed counts.
`step11_tradeoff.png`	Yield-health vs pesticide use	Up is more simulated health; left is less chemical. Reward determines which balance is preferred.
`step12_dqn.png`	DQN comparison graphic	Check it against source/table caveat before presenting.

How to describe a plot aloud

```
"This graph summarizes [metric] for [specific policies] under [specific scenario]. Each
value is from [30 or 100] simulated seasons. It suggests [narrow observation]. It does not
prove [real-world claim]."
```

12. Important limits and items to verify

A. The environment is a simulator

- No real drone or sprayer is controlled, and no field trial is reported in the checked-in output.
- The disease growth, spread, crop damage, pesticide efficacy and residue equations are assumed. They require agronomic calibration and field validation.
- Final "yield %" is average simulated cell health. It is not measured harvest weight, revenue, or crop-loss avoided in a real farm.

B. Observation classifier scope and metrics

- The saved classifier is a healthy-versus-rust cell classifier; its output is used as a rust-probability score.
- A classifier score is not the simulated cell's true severity. The simulator samples scores conditionally from held-out class score banks.
- The script prints test metrics, but no realized metric log was found among inspected artifacts. Do not repeat unsupported AUC/accuracy values from a slide unless you rerun and record them.

C. Units and thresholds

- The code calls pesticide use "dose units"; there is no verified conversion to litres. Use "units."
- The core defaults are wind > 20 km/h and rain > 0.2 mm. The source calls these assumptions. Cross-check any deck or report that states different gates, then make the wording consistent.
- The reward weights are design choices. Weight sensitivity tests recompute reward for frozen policies; they do not prove how newly retrained policies would rank under new weights.

D. Experiment strength and graph provenance

- Three PPO seeds are better than one but still a modest sample. The 0.1 and 0.0 shaping variants use two seeds and 300k steps versus three seeds and 600k for the 0.3 group.
- The 30-season and 100-fresh-season protocols are separate. Always state which one a result comes from.
- The generic `plot_evaluation_graphs.py` contains hard-coded values and a synthetic randomized cumulative-regret panel. Do not present that panel as measured regret.
- The "No State-History" figure entry is not backed by the reviewed Step 10 CSV or script. Do not claim it as an executed ablation unless its raw run/output is supplied.

E. Reproducibility issues to fix

- `dqn.py` appears to call `SmartThreshold(tau=0.15)`, but the constructor expects `cuts` and `w`; correct the call and regenerate the table or clarify the table's separate source.

- `make_demo.py` contains an absolute Windows path to a generator file that is not in the archive; make it relative and include the dependency if you want a reproducible demo build.
- The original raw drone-image folders were not included in the reviewed project copy, so the early image-preparation steps cannot be rebuilt from that archive alone.

A strong, honest defense answer

"The policy performs well under this simulator's assumptions and our chosen reward. Because important pest and treatment dynamics are modeled rather than field-calibrated, this is a proof of concept. The next step is to validate the equations and sensor pipeline with agronomists and real field trials."

13. How to reproduce the workflow

Run from the project root so relative paths such as `data/tiles/score_bank.npz` resolve correctly. The scripts contain `# %%` notebook-cell markers: the author intended them to be run cell-by-cell in an editor/notebook. Running a whole script executes every cell in order and may trigger API requests, preprocessing, or long model training.

What must be available first

- Python environment and packages. The setup check looks for NumPy, pandas, Matplotlib, Pillow and Gymnasium. The image classifier additionally uses scikit-learn and joblib. PPO/DQN training uses PyTorch and Stable-Baselines3.
- Original UAV image dataset under the path expected by `field_dataset.py`, if rebuilding from raw photos.
- Internet access to query Open-Meteo if the requested cached weather CSV is missing. No API key is required by the code.
- Enough time for training: Step 8/9/DQN request up to 600,000 decisions per run. A short smoke test is explicitly included in PPO to estimate speed first.

Dependency-aware run order

1. 1. Run `setup.py` and confirm the Python/library/API checks pass.
2. 2. Obtain the UAV image dataset, then run `field_dataset.py`.
3. 3. Run `weather.py` to fetch or verify the cached weather CSVs.
4. 4. Run `field_grid.py` to write per-cell features and visual checks.
5. 5. Run `sensor.py` to train the classifier and create the held-out score bank.
6. 6. Run `simulator.py` to validate the environment and inspect example episodes.
7. 7. Run `baselines.py` and `tank_check.py` before training RL.
8. 8. Run `train_ppo.py`; do the 20k smoke test before the full run.
9. 9. Run `robust.py` to repeat PPO seeds and compare with the stronger context rule.
10. 10. Run `shaping.py` to compare shaping settings, noting the unequal budgets.
11. 11. Run `tradeoff.py` for its separate 100-fresh-season analysis.
12. 12. Fix the DQN constructor/evaluation mismatch, then run `dqn.py` and regenerate its table.
13. 13. Make the demo generator path portable; then run `make_demo.py` only after its dependency is present.

Do not overwrite the evidence silently

If you change thresholds, reward weights, tank, random seeds, dataset splits, or simulator equations, save a new output table and label it as a new experiment. Otherwise it becomes impossible to know which settings produced the slide numbers.

14. A simple presentation script and likely questions

A 60-second explanation you can practice

Short script

"Our project studies adaptive pesticide spraying with reinforcement learning. We built an 8×8 soybean-field simulator. It combines hourly weather data and score samples from a classifier trained on drone-image cell features, while crop and disease dynamics are modeled assumptions. At each cell the agent sees noisy local observations, crop health, weather and tank level, then chooses no, low, medium or high spray. The reward encourages simulated crop-health preservation and penalizes chemical use, unnecessary spraying and a wind-based drift proxy. We trained PPO and compared it with rules and DQN. In our 100-fresh-season table, PPO improved reward over no spray and used less chemical than fixed-medium spraying, but a stronger hand-built rule remained competitive. These results are simulator evidence, not proof of field performance."

Questions you may be asked

Question	Clear answer
Why use reinforcement learning?	Because decisions are sequential: spraying now changes later infestation, residue, resources and health. RL can learn a policy from repeated feedback. We still compare it against simple rules.
Why use an 8×8 grid?	It is a manageable abstraction: enough cells to model spatial hotspots, but small enough to run many simulated episodes. It is not a claim that a real farm is 64 cells.
Does the model actually detect pests?	The saved classifier is trained for healthy versus rust cell scores. Its output is only a noisy simulator observation; the simulator's hidden infestation is not a direct live model output.
What does "yield" mean?	The final mean cell-health percentage in the simulation. It is not crop mass or harvest data.
Why are rewards negative?	Reward includes health change and several penalties. Negative absolute values are possible; policy gains compare one policy's reward with no spray in the same scenario.

Question	Clear answer
Did PPO beat the heuristic?	Not clearly in every test. The 30-season robust summary says no clear difference on seen conditions and shows heuristic advantages in some scenarios. The 100-season table gives PPO a small +0.14 reward-point edge over context, but PPO uses more chemical.
Can this be used on a real farm now?	No. The treatment/disease dynamics and limits need agronomic calibration, validation, safety review and real field testing.
What is the biggest next step?	Reconcile script/table mismatches, log actual classifier metrics, validate simulator parameters with domain experts, and test on independently collected field data before any deployment claim.

15. Project file map and glossary

Where to look in the project

Path / file	What it contains
`setup.py`	Machine, package, folder and API checks.
`field_dataset.py`	Image selection, resizing and class-folder preparation.
`weather.py`, `src/weather_api.py`	Real weather download/cache and location selection.
`field_grid.py`	8×8 partitioning and the tile-feature CSV creation.
`sensor.py`	Classifier candidates, validation selection, score bank and sensor plot.
`src/spray_env.py`	Core simulator: observation, actions, constraints, field updates, rewards.
`src/policies.py`	No-spray, fixed, threshold, context and oracle policies.
`simulator.py`	Environment checker, example episodes and sanity figures.
`baselines.py`, `tank_check.py`	Baseline tuning/evaluation and tank-capacity experiments.
`train_ppo.py`, `dqn.py`	PPO and DQN definitions/training/evaluation.
`robust.py`, `shaping.py`, `tradeoff.py`	Repeated-seed robustness, reward-shaping ablation, and fresh-season trade-offs.
`outputs/*.csv`	Tables and episode logs; use these to verify plotted values.
`outputs/*.png`	Step-specific original project figures.
`graphs/*.png`, `plot_evaluation_graphs.py`	Report-style chart set; audit its data provenance before treating every panel as measured.

Path / file	What it contains
`make_demo.py`, `outputs/*.html`	Demo generator and saved HTML artifacts; generator has a path portability issue.

Glossary of project-specific measures

Measure	How this guide uses the term
Reward	The project's weighted score for an episode. Units are arbitrary simulated reward points.
Gain vs no spray	Mean policy reward minus mean no-spray reward in the same scenario/evaluation set.
Yield %	100 × mean final simulated cell health; not actual harvested crop.
Pesticide units	Sum of simulated dose units. No verified conversion to litres is provided.
Unnecessary units	Dose applied when hidden pre-spray infestation severity is below $\tau = 0.15$.
Drift units	Dose weighted by a clipped/normalized wind measure; it is a simple modeled penalty.
Seed spread	Reported variation across separately trained PPO seeds in the relevant output, not an exact confidence interval.
Fresh season	New simulated episode seed set used by the Step 11 script; it is not a new real farm trial.

Final takeaway

What the project successfully demonstrates

A complete, inspectable reinforcement-learning workflow: image-derived observation scores, real hourly weather input, a custom partially observable farm simulator, rule baselines, PPO training, scenario testing, ablations, trade-off analysis and visual outputs. The strongest next contribution is improving reproducibility and validating assumptions—not claiming field-ready spraying.

End of guide. When presenting, keep the story simple: explain what the agent observes, what choices it makes, how the reward balances goals, what the strongest comparison showed, and which parts still need validation.